



IronHook

基于 Rust 的 Android AArch64 Inline Hook 框架

结题汇报

李安峤 钟启帆 范宇祺 熊逸飞 梁嘉文



汇报结构

1. **背景与目标**: Android Native Hook 的安全风险、Rust 重构动机、与 ByteDance Android inline hook 的定位差异。
2. **系统设计**: Rust Core、C ABI、ELF 符号解析、trampoline、内存权限与 cache flush。
3. **核心实现**: AArch64 PC-relative 指令重写、hook 生命周期、错误处理和工程产物。
4. **验证结果**: 主机测试、Android 构建、QEMU 端到端 smoke test。
5. **总结与展望**: V1 完成情况、主动收缩的边界、后续工作。

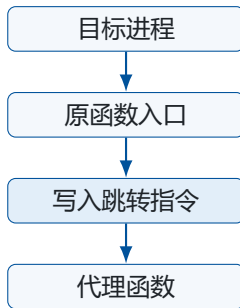
本次汇报重点放在 inline hook 核心链路: 函数入口 patch、trampoline、AArch64 指令重写, 以及如何用 Rust 约束这些高风险操作。

项目背景：为什么需要 Inline Hook

- Android Native 层常用于 APM、安全防护、非侵入调试和逆向分析。
- Inline Hook 通过运行时修改函数入口指令，将控制流重定向到代理函数。
- 它不依赖源码修改，也不要求重新链接原程序。

关键风险

改写的是正在被 CPU 执行的机器码，任何越界写入、错误重定位或 cache 不一致，都可能导致宿主进程崩溃。



对标对象: ByteDance Android Inline Hook

维度	ByteDance Android inline hook / ShadowHook 思路	IronHook V1
语言与生态	C/C++ 实现, 贴近 Android Native 使用习惯, 功能成熟。	Rust 核心实现, C ABI 对外暴露, 重点验证 Rust 系统编程可行性。
支持范围	工业级功能更完整, 支持更多 hook 模式和 Android 场景。	聚焦 Android AArch64 unique inline hook, 不追求一次覆盖全部能力。
指令修复	工程经验丰富, 但宏、位运算和分支路径维护门槛高。	使用枚举和模式匹配组织 AArch64 指令类型, 便于单元测试。
错误模型	C 风格返回值和 errno。	Rust Result 内部传播, C ABI errno 对外兼容。

我们的定位: 不复刻完整 ShadowHook, 而是把函数入口 patch、trampoline、AArch64 重写、Android 构建和 QEMU 验证这条核心链路跑通。



为什么选择 Rust 重构核心链路

C/C++ 版本常见风险

裸指针散落、错误恢复路径不统一、生命周期依赖人工约束、并发状态难审计。

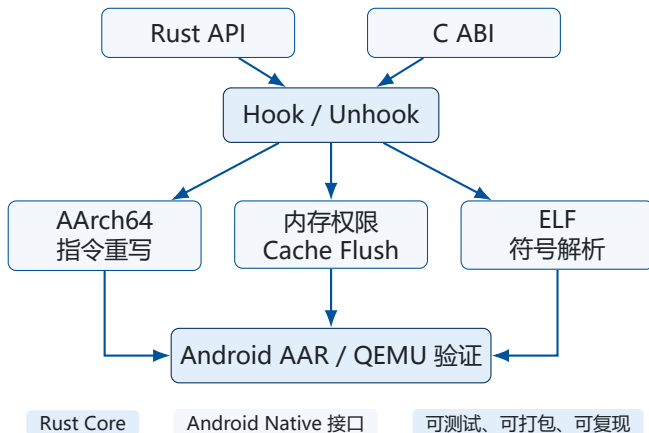
Hook 场景无法逃避 unsafe

mprotect、裸地址、机器码写入和内联汇编必然存在，但可以收束边界。

Rust 带来的收益

- Result<T, E> 强制处理失败路径
- RAII 管理 hook 生命周期
- enum + match 表达指令状态空间
- unsafe 集中在少量模块中审计

总体架构





仓库结构与模块职责

- ▶ `api.rs`: Rust API, 地址 hook 和符号 hook。
- ▶ `hook.rs`: 安装、patch、卸载主流程。
- ▶ `arch/a64.rs`: AArch64 指令分类与重写。
- ▶ `memory.rs`: `mmap`、`mprotect`、跨页写入、`cache flush`。
- ▶ `elf.rs`: 进程内 ELF 符号解析。
- ▶ `ffi.rs`: C ABI、`errno`、兼容别名。
- ▶ `android/ironhook`: Android AAR 打包。
- ▶ `examples / scripts`: QEMU 与构建验证。

核心代码、示例、脚本和文档约 4965 行。

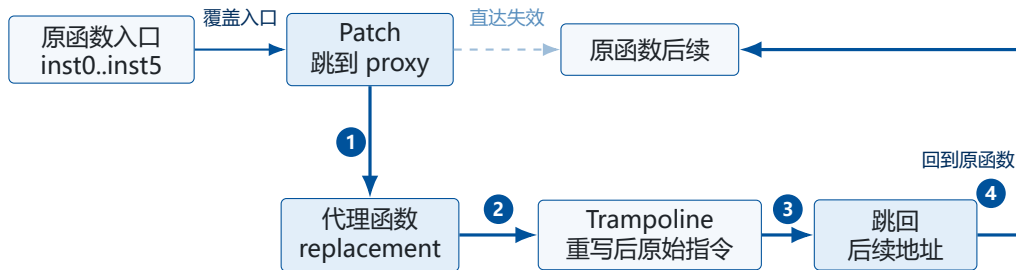


Hook 安装流程

1. 参数检查：目标函数地址和代理函数地址非空，满足 AArch64 4 字节对齐。
2. 重复检查：unique 模式下同一目标地址只能安装一个 hook。
3. 备份入口：复制目标函数入口处固定长度的原始指令。
4. 指令分析：计算每条指令搬运到 trampoline 后的重写长度。
5. trampoline 构建：写入重写后的原始指令，并在尾部跳回原函数后续地址。
6. 目标 patch：临时修改代码页权限，写入跳到代理函数的绝对跳转。
7. 状态记录：保存 backup、patch bytes、trampoline 地址，用于 unhook。

失败路径全部返回错误：参数错误、重复 hook、内存分配失败、mprotect 失败、指令重写失败等。

Trampoline: 保留原函数语义



目标函数入口被 patch 后，直接调用原函数会再次进入代理函数。代理函数先跳入 trampoline；trampoline 执行被覆盖指令的等价路径，再回到原函数后续地址，从而保留原始语义。



AArch64 指令重写：分类与策略

- ▶ 普通指令：直接复制。
- ▶ 分支指令：计算原目标地址，改写为绝对跳转。
- ▶ 地址生成：ADR/ADRP materialize 目标地址。
- ▶ 条件分支：反转条件 + 跳过绝对跳转。
- ▶ 不安全场景：返回 RewriteFailed。

```
match classify(inst) {  
  InstType::B | InstType::BL =>  
    rewrite_b(buf, inst, pc, ctx),  
  InstType::Adr =>  
    rewrite_adr(buf, inst, pc, ctx),  
  InstType::Adrp =>  
    rewrite_adrp(buf, inst, pc, ctx),  
  InstType::Ignored => {  
    buf.write(inst);  
    4  
  }  
}
```

AArch64 指令重写：支持范围

指令类型	处理方式	风险点
B / BL	计算目标地址，生成 LDR X17 + BR/BLR X17。	分支目标可能落在 backup 区间内部。
B.cond	保留条件语义，局部跳过绝对跳转序列。	条件反转和偏移计算容易出错。
ADR / ADRP Literal load	将 PC-relative 地址改写为绝对地址加载。 先加载绝对地址，再按原语义读取数据。	ADRP 33 位符号扩展。 内部 literal 数据引用不安全时拒绝。
CBZ/TBZ 系列	条件满足时跳到 materialized 地址。	内部目标需要映射到 trampoline 地址。

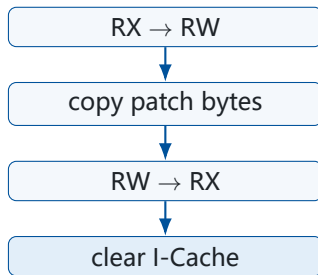
内存权限：避免长期 W+X

Trampoline

mmap 分配 RW 内存，写入重写后的指令，完成后切换为 RX。

目标函数 patch

计算覆盖页范围，临时切换为 RW，写入后立刻恢复 RX。



测试覆盖了跨页 patch 范围，避免只在单页场景下成立。



Cache Flush: 让 CPU 看到新指令

- ARM 处理器常见实现中 I-Cache 和 D-Cache 分离。
- 写入 patch 后, 新指令最初可能只在 D-Cache 中。
- 如果不失效 I-Cache, CPU 取指可能仍读到旧机器码。

IronHook 在 AArch64 上读取 `ctr_el0` 获取 cache line 大小, 对写入范围执行 D-cache clean、I-cache invalidate, 并使用 `dsb / isb` 屏障同步流水线。



ELF 符号解析：按已加载库查找函数

- ▶ 通过进程 map 和 ELF program header 定位已加载库。
- ▶ 解析 dynamic section、symbol table、string table。
- ▶ 支持 `resolve_symbol("libc.so", "puts")`。
- ▶ 支持 `resolve_addr(addr)` 反查符号。

dev 分支增强：结合 goblin 的 Ctx 和 scroll 按 ELF 上下文解析 program header；GNU hash 解析补充 bloom 参数校验和 symbol count 估算，提升 Android 真实进程内存布局下的可靠性。



对外接口: Rust API 与 C ABI

Rust API

```
let sym = resolve_symbol("libc.so", "puts")?;  
let (guard, trampoline) =  
    HookBuilder::new(&sym, proxy_addr)  
        .install()?;  
  
let original = unsafe { trampoline.as_fn() };
```

C ABI

```
void *orig = NULL;  
void *stub = ironhook_hook_func_addr(  
    target, replacement, &orig);  
  
if (stub == NULL) {  
    int err = ironhook_get_errno();  
}
```

Rust 内部保留类型系统和错误表达能力; Android/NDK 集成方仍可使用稳定 C ABI。



验证覆盖

验证层级	覆盖内容
Host checks	fmt、clippy、workspace tests、compat-shadowhook feature tests。
AArch64 rewrite	ADRP 符号扩展、内部跳转修正、literal load 拒绝、条件分支、向前/向后跳转。
Memory tests	零大小分配拒绝、非对齐地址、跨页 patch 写入。
Android build	aarch64-linux-android release 产物, AAR 中 libironhook.so 与 ironhook.h 检查。
QEMU smoke test	puts hook、trampoline、unhook、重复 hook 拒绝、Rust API 和 C ABI。



Demo: 拦截 Android libc.so 的 puts

1. `resolve_symbol("libc.so", "puts")` 找到符号地址。
2. 安装 hook, 函数入口跳到 `puts_proxy`。
3. 代理函数返回自定义结果。
4. trampoline 调用原始 `puts`。
5. guard drop 后自动 unhook。

```
=== IronHook Android Demo ===  
symbol resolver verified  
hook installed (guard active)  
duplicate hook rejected  
verified: proxy no longer called after unhook  
verified: reinstall and automatic unhook passed  
=== All tests passed ===
```



工程产物

可使用产物

- Rust crate: `ironhook`
- Android so: `libironhook.so`
- Android AAR:
`ironhook-android-release.aar`
- Header: `ironhook.h`

可复现文档

- `README.md`
- `DEV_GUIDE.md`
- `docs/API.md`
- `docs/VALIDATION.md`



完成情况

已经完成

- Android AArch64 unique inline hook
- 地址 hook / 已加载符号 hook
- trampoline 原函数调用
- unhook 恢复
- Rust API / C ABI / AAR
- QEMU 端到端验证

暂未实现

- Arm32 / Thumb
- shared / multi hook 链
- pending newly loaded ELF hook
- linker init / fini callback
- 并发安装/卸载压力测试



与前期计划的差异

- ▶ 前期考虑引入 Capstone; 最终 V1 选择手写 AArch64 分类器, 减少依赖并提高测试粒度。
- ▶ 前期规划 shared / multi 等完整 hook 模式; 最终主动收缩到 unique 模式, 优先保证核心链路稳定。
- ▶ 前期计划真机验证; 最终形成了主机测试、Android 构建和 QEMU user-mode smoke test 的可复现链路。

项目取舍原则: 先把最危险、最核心的部分做成可解释、可测试、可失败恢复的原型。



后续工作

1. 补充并发调用与安装/卸载压力测试。
2. 加固 hook manager 的全局临界区，规避竞态窗口。
3. 扩展 AArch64 指令重写覆盖范围。
4. 增加 Android NDK consumer 示例，验证 AAR 集成流程。
5. 进一步细化错误诊断和 debuggable 日志。



总结

- IronHook 验证了用 Rust 实现 Android AArch64 inline hook 核心链路的工程可行性。
- 项目重点不是复刻完整 ShadowHook，而是跑通最小但完整的系统级闭环。
- Rust 带来的收益主要体现在 unsafe 边界收束、结构化错误处理、指令重写逻辑可测试。
- 当前 V1 仍是实验原型，后续要补齐并发、更多指令、更多 hook 模式和真机验证。



谢谢大家

Q & A